

Working Memory Layer

Implementation Plan — Mapped to Your Actual Codebase

Gaurav Kumar · June 11, 2026 · v1.0 · Based on system report + architecture review

This document bridges the gap between the working memory architecture (what to build) and your actual code (where to build it). Every task is mapped to a specific file and line. Follow the sprint order — each task depends on the previous one.

0 — Current System — What You Have Right Now

READ THIS FIRST

What already works (do not break this)

- ✓ Full message history is sent every turn — ChatPage.jsx line 885: nextMessages = [...messages, userMessage]
- ✓ Backend passes the full array to the LLM — chat.py line 56: provider_instance.generate_stream(model_name, messages_list)
- ✓ All providers (OpenAI, Anthropic, Gemini) receive the correct messages array
- ✓ SSE streaming already works end-to-end

What is broken / missing right now

- No rolling limit — you send ALL messages every turn. At 50+ messages this will hit token limits or cost a lot
- memory_working.py has broken routes — /api/config and /api/verify-provider duplicate auth.py routes
- memory_working.py references PROVIDER_MODULES and get_provider_module which are not defined — will crash if hit
- use_memory, memory_provider, session_id fields are sent from frontend but completely ignored by chat.py
- No rolling_summary field exists anywhere yet — neither in React state nor in the backend request model
- No compression logic exists — chat.py does not compress or truncate anything
- No chat persistence — refresh = lost history

Files you will touch across all sprints

File	Location	What changes
ChatPage.jsx	frontend/src/	Add rolling_summary to session state, add compression trigger, handle SSE control frame
chat.py	backend/	Add rolling_summary to ChatRequest, add token pre-flight, wire async compression fork
memory_working.py	backend/	Fix broken routes, add /api/compress endpoint for Memory LLM call
NEW: memory_utils.py	backend/	tiktoken budget checker, compression trigger logic, anchored prompt builder
NEW: compression_service.py	backend/	Async Memory LLM call, grounding pass logic, summary storage

Goal

Stop sending the full unbounded message array to the LLM. Cap it at 20 messages. This is the minimum viable fix — without it the app breaks on long conversations. This sprint requires zero new files and touches only two places.

Task 1.1 — Add rolling_summary to session state

File: frontend/src/ChatPage.jsx · Line ~37 (createEmptySession)

```
// BEFORE (line ~37): const createEmptySession = () => ({ id: generateId(), messages: [], // ...other fields
}); // AFTER — add rolling_summary: const createEmptySession = () => ({ id: generateId(), messages: [],
rolling_summary: "", // <-- ADD THIS // ...other fields });
```

Task 1.2 — Cap messages to last 20 before sending

File: frontend/src/ChatPage.jsx · Line ~918 (the fetch/POST block)

```
// BEFORE (line ~918): body: JSON.stringify({ provider: provider.provider, api_key: provider.apiKey,
model_name: provider.modelName, ...getMemoryPayloadFields(), messages: nextMessages.map(({ role, content })
=> ({ role, content })), }) // AFTER — rolling window of 20 + include rolling_summary: const ROLLING_LIMIT =
20; const windowMessages = nextMessages.slice(-ROLLING_LIMIT); body: JSON.stringify({ provider:
provider.provider, api_key: provider.apiKey, model_name: provider.modelName, rolling_summary:
activeSession.rolling_summary, // <-- ADD messages: windowMessages.map(({ role, content }) => ({ role,
content })), })
```

Task 1.3 — Accept rolling_summary in backend ChatRequest

File: backend/chat.py · Line ~15 (ChatRequest model)

```
# BEFORE (line ~15): class ChatRequest(BaseModel): provider: str api_key: str model_name: str messages:
list[Message] # memory fields that are currently ignored... # AFTER — add rolling_summary: class
ChatRequest(BaseModel): provider: str api_key: str model_name: str messages: list[Message] rolling_summary:
str = "" # <-- ADD, default empty string
```

Task 1.4 — Prepend rolling_summary to messages before LLM call

File: backend/chat.py · Line ~52 (messages_list construction)

```
# BEFORE (line ~52): messages_list = [{"role": msg.role, "content": msg.content} for msg in req.messages] #
AFTER — inject rolling_summary as first user/system message if present: messages_list = [{"role": msg.role,
"content": msg.content} for msg in req.messages] if req.rolling_summary: summary_message = { "role":
"user", "content": f"[CONVERSATION CONTEXT - earlier messages summarised]\n{req.rolling_summary}" } #
Insert after system prompt (index 0) or at start if no system prompt insert_at = 1 if messages_list and
messages_list[0]["role"] == "system" else 0 messages_list.insert(insert_at, summary_message)
```

Sprint 1 done — test checklist

- Send 25 messages in one session — backend should only receive 20
- rolling_summary is empty string for now — that is correct, compression comes in Sprint 2
- LLM still has full recent context (last 20 messages)
- No existing tests should break — this is additive only

Goal

Message count is a bad proxy for tokens. A single pasted code block can be 800 tokens. Before every LLM call, measure actual tokens and drop oldest messages if over budget. This prevents hard API errors. Build this BEFORE the async compression fork — you need the budget logic to work standalone first.

Task 2.1 — Create memory_utils.py

File: backend/memory_utils.py · NEW FILE

```
# backend/memory_utils.py
import tiktoken # Use cl100k_base as universal fallback — works for GPT-4, Claude, Gemini estimates
def get_encoder():
    try:
        return tiktoken.encoding_for_model("gpt-4o")
    except Exception:
        return tiktoken.get_encoding("cl100k_base")
enc = get_encoder()
def count_tokens(text: str) -> int:
    return len(enc.encode(text))
def build_payload_within_budget(
    system_prompt: str,
    rolling_summary: str,
    messages: list[dict],
    model_limit: int = 128000,
    output_reserve: int = 2000,
) -> dict:
    budget = model_limit - output_reserve # Non-negotiable — always include used = count_tokens(system_prompt) + count_tokens(rolling_summary)
    if used > budget: # Emergency: summary itself too large, trigger grounding pass
        raise ValueError(f"Summary exceeds budget ({used} tokens). Trigger grounding pass.")
    # Fill from newest message backwards
    messages_to_send = []
    for msg in reversed(messages):
        cost = count_tokens(msg["content"]) + 4 # 4 = per-message overhead
        if used + cost > budget:
            break
        messages_to_send.insert(0, msg)
        used += cost
    # Hard floor: always keep last 5 messages regardless if len(messages_to_send) < 5 and len(messages) >= 5:
    messages_to_send = messages_to_send[-5:]
    return {
        "messages": messages_to_send,
        "tokens_used": used,
        "tokens_remaining": budget - used,
        "messages_dropped": len(messages) - len(messages_to_send),
    }
def should_compress(messages: list, trigger_threshold: int = 30) -> bool:
    return len(messages) >= trigger_threshold
def get_chunk_for_compression(messages: list, R: int = 10) -> tuple[list, list]:
    """Returns (chunk_to_compress, raw_buffer_to_keep)"""
    if len(messages) <= R:
        return [], messages
    return messages[:-R], messages[-R:]
```

Task 2.2 — Wire budget check into chat.py

File: backend/chat.py · Line ~52 (replace messages_list construction)

```
# ADD import at top of chat.py: from memory_utils import build_payload_within_budget, count_tokens
# REPLACE the messages_list block (line ~52):
raw_messages = [{"role": msg.role, "content": msg.content} for msg in req.messages]
# Inject rolling summary if present
if req.rolling_summary:
    summary_msg = {"role": "user", "content": f"[CONTEXT]\n{req.rolling_summary}"}
    insert_at = 1
    if raw_messages and raw_messages[0]["role"] == "system":
        raw_messages.insert(insert_at, summary_msg)
    # Pre-flight token budget check
    system_prompt = next((m["content"] for m in raw_messages if m["role"] == "system"), "")
    budget_result = build_payload_within_budget(
        system_prompt=system_prompt,
        rolling_summary=req.rolling_summary,
        messages=[m for m in raw_messages if m["role"] != "system"],
    )
    messages_list = budget_result["messages"]
    # Log if messages were dropped (useful for debugging)
    if budget_result["messages_dropped"] > 0:
        print(f"[budget] Dropped {budget_result['messages_dropped']} messages to fit context window")
```

Sprint 2 done — test checklist

- Paste a 500-line code block as a message — should not crash, should log 'Dropped N messages'
- Normal conversation of 20 messages — 0 dropped, all messages pass through
- rolling_summary still empty string — that is fine

Goal

When messages hit the trigger threshold ($T=30$), fire an async compression job in parallel with the main LLM response. When compression finishes, inject a control frame into the SSE stream. The frontend silently updates its state. Zero latency added to the user.

Task 3.1 — Create `compression_service.py`

File: `backend/compression_service.py` · [NEW FILE](#)

```
# backend/compression_service.py
import asyncio, json from memory_utils import count_tokens
ANCHORED_COMPRESSION_PROMPT = """ You are a memory manager. Your job is NOT to rewrite the existing summary.
Your job is to EXTEND it with new information from the conversation chunk. Rules: 1. Preserve every fact,
name, path, variable, and decision from the existing summary EXACTLY as written. 2. Only add new information
from the conversation chunk below. 3. If new info contradicts an existing fact, update only that line:
[UPDATED: old value -> new value] 4. Do NOT paraphrase existing content. Do NOT restructure. Output format:
--- PRESERVED SUMMARY --- {existing_summary} --- NEW ADDITIONS --- [only new facts, decisions, preferences,
constraints from chunk] --- ENTITIES (new in this chunk) --- Files/Paths: [list or NONE]
Variables/Functions: [list or NONE] Decisions made: [list or NONE] User preferences: [list or NONE]
Errors/Bugs: [list or NONE] """
async def compress_chunk( chunk_messages: list[dict], existing_summary:
str, provider_instance, model_name: str, max_summary_tokens: int = 800, ) -> str:
    prompt = ANCHORED_COMPRESSION_PROMPT.format( existing_summary=existing_summary or "(no prior summary)" )
    chunk_text = "\n".join( f"[{m['role'].upper()}]: {m['content']}" for m in chunk_messages )
    messages = [ {"role": "user", "content": f"{prompt}\n\nCONVERSATION CHUNK:\n{chunk_text}"} ]
    # Use cheaper model for compression
    result_tokens = []
    async for chunk in provider_instance.generate_stream(model_name, messages):
        result_tokens.append(chunk)
    new_summary = "".join(result_tokens)
    # Cap summary at max_summary_tokens
    if count_tokens(new_summary) > max_summary_tokens:
        # Trim by truncating new additions section only
        lines = new_summary.split("\n")
        token_count = 0
        for line in lines:
            lc = count_tokens(line)
            if token_count + lc > max_summary_tokens:
                trimmed.append("[summary capped at token limit]")
                break
            trimmed.append(line)
            token_count += lc
    new_summary = "\n".join(trimmed)
    return new_summary
```

Task 3.2 — Add async fork to chat.py stream endpoint

File: `backend/chat.py` · [Modify the streaming endpoint](#)

```
# ADD imports at top: from compression_service import compress_chunk from memory_utils import
should_compress, get_chunk_for_compression
import asyncio, json # MODIFY the streaming endpoint to fork
when compression is needed:
@app.post("/api/chat")
async def chat_endpoint(req: ChatRequest):
    async def event_generator():
        # --- Check if compression should fire ---
        all_messages = [{"role": m.role, "content": m.content} for m in req.messages]
        compression_task = None
        if should_compress(all_messages, trigger_threshold=30):
            chunk, raw_buffer = get_chunk_for_compression(all_messages, R=10)
            if chunk:
                # Fire compression in background - do NOT await here
                compression_task = asyncio.create_task( compress_chunk(
                    chunk_messages=chunk, existing_summary=req.rolling_summary,
                    provider_instance=provider_instance, model_name=model_name, ) )
            # --- Task A: Stream main response to user (unchanged) ---
            async for token in provider_instance.generate_stream(model_name, messages_list):
                yield f"data: {json.dumps({'text': token})}\n\n"
            # --- After streaming: check if compression finished ---
            if compression_task is not None:
                try:
                    new_summary = await asyncio.wait_for(compression_task, timeout=30.0)
                except:
                    raw_buffer = get_chunk_for_compression(all_messages, R=10)
                    control_frame = { "control": "memory_compact", "rolling_summary": new_summary, "truncated_count": len(all_messages) - len(raw_buffer) }
                    yield f"data: {json.dumps(control_frame)}\n\n"
                except asyncio.TimeoutError:
                    # Compression too slow - skip this round, try next trigger pass
                    except Exception as e:
                        # Never crash main chat for compression failure
                        print(f"[compression] Failed: {e}")
            return EventSourceResponse(event_generator())
```

Task 3.3 — Handle control frame in React frontend

File: `frontend/src/ChatPage.jsx` · [SSE message handler \(line ~865 area\)](#)

```
// In your SSE / EventSource message handler, ADD control frame handling:
eventSource.onmessage = (e) => {
    const data = JSON.parse(e.data);
    // Handle memory compression control frame
    if (data.control === "memory_compact") {
        setActiveSession(prev => ({ ...prev, rolling_summary: data.rolling_summary, // Prune old messages from the payload window (UI still shows all) messages:
        prev.messages.slice(data.truncated_count), }));
        return; // don't treat this as a text token
    } // Normal token - append to current assistant message bubble
    if (data.text) {
        setCurrentMessage(prev => prev + data.text);
    }
};
```

Sprint 3 done — test checklist

- Send 32 messages — compression should fire on message 31
 - SSE stream delivers tokens normally — no delay
 - After stream ends, control frame arrives — `rolling_summary` is populated in React state
 - On message 32, backend receives `rolling_summary` as non-empty string
 - Check browser devtools Network tab — control frame visible in SSE event stream
-

Goal

After 5–6 compression rounds, summaries drift. The anchored prompt (already in Sprint 3) handles 80% of this. The grounding pass fixes the remaining 20% by reconciling the last 3 summary versions every 5th compression. This sprint also adds a compression counter to track when to fire the grounding pass.

Task 4.1 — Track compression epoch in React state

File: `frontend/src/ChatPage.jsx` · `createEmptySession` (~line 37)

```
// ADD to createEmptySession: const createEmptySession = () => ({ id: generateId(), messages: [],
rolling_summary: "", compression_epoch: 0, // <-- ADD: counts compression rounds summary_history: [], //
<-- ADD: keeps last 3 summaries for grounding }); // In the control frame handler, also update epoch and
history: if (data.control === "memory_compact") { setActiveSession(prev => { const newHistory = [
...prev.summary_history.slice(-2), // keep last 2 prev.rolling_summary, // add current before replacing
].filter(Boolean); return { ...prev, rolling_summary: data.rolling_summary, messages:
prev.messages.slice(data.truncated_count), compression_epoch: prev.compression_epoch + 1, summary_history:
newHistory, }; }); return; }
```

Task 4.2 — Send epoch + history to backend

File: `frontend/src/ChatPage.jsx` · `fetch POST block` (~line 918)

```
body: JSON.stringify({ provider: provider.provider, api_key: provider.apiKey, model_name:
provider.modelName, rolling_summary: activeSession.rolling_summary, compression_epoch:
activeSession.compression_epoch, // <-- ADD summary_history: activeSession.summary_history, // <-- ADD
messages: windowMessages.map(({ role, content }) => ({ role, content })), })
```

Task 4.3 — Add grounding pass to `compression_service.py`

File: `backend/compression_service.py` · `Add grounding function`

```
# ADD to compression_service.py: GROUNDING_PROMPT = """ You are given 2-3 versions of a conversation
summary (oldest to newest). Produce ONE canonical summary that: - Keeps the most recent version of any fact
that changed - Removes contradictions between versions - Preserves ALL unique identifiers, file paths,
decisions, and constraints - Does NOT add any new information - Output only the final canonical summary, no
explanations Summary versions (oldest first): {versions} """ async def grounding_pass( summary_history:
list[str], current_summary: str, provider_instance, model_name: str, ) -> str: all_versions = [s for s in
summary_history if s] + [current_summary] if len(all_versions) < 2: return current_summary # nothing to
reconcile versions_text = "\n\n---\n\n".join( f"Version {i+1}: {s}" for i, s in enumerate(all_versions) )
prompt = GROUNDING_PROMPT.format(versions=versions_text) messages = [{"role": "user", "content": prompt}]
result = [] async for chunk in provider_instance.generate_stream(model_name, messages):
result.append(chunk) return "".join(result)
```

Task 4.4 — Trigger grounding pass every 5th compression in `chat.py`

File: `backend/chat.py` · `In the compression task block (Sprint 3 code)`

```
# MODIFY the compression block in event_generator(): if compression_task is not None: try: new_summary =
await asyncio.wait_for(compression_task, timeout=30.0) # Every 5th compression: run grounding pass to fix
drift epoch = getattr(req, 'compression_epoch', 0) if epoch > 0 and epoch % 5 == 0: summary_history =
getattr(req, 'summary_history', []) new_summary = await grounding_pass( summary_history=summary_history,
current_summary=new_summary, provider_instance=provider_instance, model_name=model_name, ) _, raw_buffer =
get_chunk_for_compression(all_messages, R=10) control_frame = { "control": "memory_compact",
"rolling_summary": new_summary, "truncated_count": len(all_messages) - len(raw_buffer), } yield f"data:
{json.dumps(control_frame)}\n\n" except Exception as e: print(f"[compression] Failed: {e}")
```

Also add to `ChatRequest` in `chat.py`

```
class ChatRequest(BaseModel): provider: str api_key: str model_name: str messages: list[Message]
rolling_summary: str = "" compression_epoch: int = 0 # <-- ADD summary_history: list[str] = [] # <-- ADD
```

Sprint 4 done — test checklist

- Run 150+ messages — check that summary quality stays consistent across 6+ compression rounds
- At epoch 5, 10, 15 — grounding pass fires, check logs
- `summary_history` in React state should always hold last 2–3 summaries

Goal

memory_working.py has duplicate routes that conflict with auth.py and references undefined variables. These will cause silent failures or 500 errors in production. Fix them now before they cause hard-to-debug issues.

Task 5.1 — Remove duplicate routes from memory_working.py

File: backend/memory_working.py

- Routes /api/config and /api/verify-provider exist in BOTH auth.py and memory_working.py — FastAPI will use whichever is registered last, unpredictably

```
# REMOVE from memory_working.py – these belong in auth.py only: # DELETE the /api/config route (line ~116) #
DELETE the /api/verify-provider route (line ~183) # ALSO FIX – remove references to undefined variables: #
DELETE or stub any usage of PROVIDER_MODULES # DELETE or stub any usage of get_provider_module() # These are
not defined anywhere and will raise NameError
```

Task 5.2 — Add the /api/compress endpoint to memory_working.py

File: backend/memory_working.py · Replace broken content with this

```
# backend/memory_working.py – clean version from fastapi import APIRouter from pydantic import BaseModel
from compression_service import compress_chunk, grounding_pass from memory_utils import should_compress,
get_chunk_for_compression router = APIRouter() class CompressRequest(BaseModel): messages: list[dict]
rolling_summary: str = "" summary_history: list[str] = [] compression_epoch: int = 0 class
CompressResponse(BaseModel): new_summary: str truncated_count: int grounding_applied: bool
@router.post("/api/compress", response_model=CompressResponse) async def compress_endpoint(req:
CompressRequest): chunk, raw_buffer = get_chunk_for_compression(req.messages, R=10) if not chunk: return
CompressResponse( new_summary=req.rolling_summary, truncated_count=0, grounding_applied=False ) return
CompressResponse( new_summary=req.rolling_summary, truncated_count=len(chunk), grounding_applied=False )
```

Sprint 5 done — test checklist

- Start the backend — zero 500 errors on startup
- GET /api/config returns correct response (from auth.py, not memory_working.py)
- POST /api/compress returns 200 with valid JSON

Sprint	Task	File	Status	Effort
1	Add rolling_summary to session state	ChatPage.jsx ~37	ADD	10 min
1	Cap messages to last 20	ChatPage.jsx ~918	MODIFY	10 min
1	Accept rolling_summary in ChatRequest	chat.py ~15	MODIFY	5 min
1	Prepend summary to messages_list	chat.py ~52	MODIFY	15 min
2	Create memory_utils.py	backend/	NEW FILE	30 min
2	Wire budget check into chat.py	chat.py ~52	MODIFY	20 min
3	Create compression_service.py	backend/	NEW FILE	45 min
3	Add async fork to chat.py stream	chat.py	MODIFY	45 min
3	Handle control frame in React	ChatPage.jsx	MODIFY	20 min
4	Track compression epoch in state	ChatPage.jsx ~37	MODIFY	15 min
4	Send epoch + history to backend	ChatPage.jsx ~918	MODIFY	10 min
4	Add grounding_pass function	compression_service.py	ADD	30 min
4	Trigger grounding every 5th round	chat.py	MODIFY	15 min
4	Add epoch/history to ChatRequest	chat.py ~15	MODIFY	5 min
5	Remove duplicate routes	memory_working.py	MODIFY	20 min
5	Fix undefined variable references	memory_working.py	MODIFY	10 min
5	Add /api/compress endpoint	memory_working.py	MODIFY	20 min

Mathematical Framework

Variable	Name	Description	Default
T	Trigger Threshold	Total messages before compression fires	30
R	Raw Buffer	Recent messages kept uncompressed	10
C	Chunk Size	Messages compressed each round ($C = T - R$)	20

Five-Phase Flow Summary

- Phase 1 — Frontend sends: rolling_summary + last R messages + new message
- Phase 2 — Backend forks: Task A streams response to user, Task B runs Memory LLM compression
- Phase 3 — SSE stream delivers tokens (Lane 1) + control frame when compression done (Lane 2)
- Phase 4 — React intercepts control frame, silently updates rolling_summary, slices messages array
- Phase 5 — Next turn starts lean: only R raw messages + updated summary in payload

Token budget priority order (tight context)

Priority	Content	Drop when tight?
1 — Never	System prompt + constraints	No
2 — Never	Rolling summary	No — more dense per token than raw messages
3 — Never	Last 5 messages	No — minimum coherence floor
4 — Drop last	Messages 6 to R	Yes — drop oldest of these first

Edge cases handled by this architecture

- Compression LLM slow → use raw history for that turn, inject summary next turn
- Compression LLM fails → log, retry at next trigger, never crash main chat
- Single message too large → summarise it individually, append [condensed] marker
- Summary grows too large → cap at 800 tokens in compression_service.py

Next document: Episodic Memory Layer — storing and retrieving past sessions with LangMem.